
Module I

OPERATING SYSTEMS

Software

- ★ Software is a **set of instructions**, data or programs used to operate computers and execute specific tasks.
- ★ Software is a generic term used to refer to **applications and programs that run on a device.**
- ★ The two main categories of software are application software and system software

System Software

- ★ System software are designed to run a computer's application programs and hardware.
- ★ System software coordinates the activities and functions of the hardware and software.
- ★ It controls the operations of the hardware and provides an environment for other softwares.
- ★ Eg: Operating system, translators, linker, loader, text editor, debugger.

Application Software

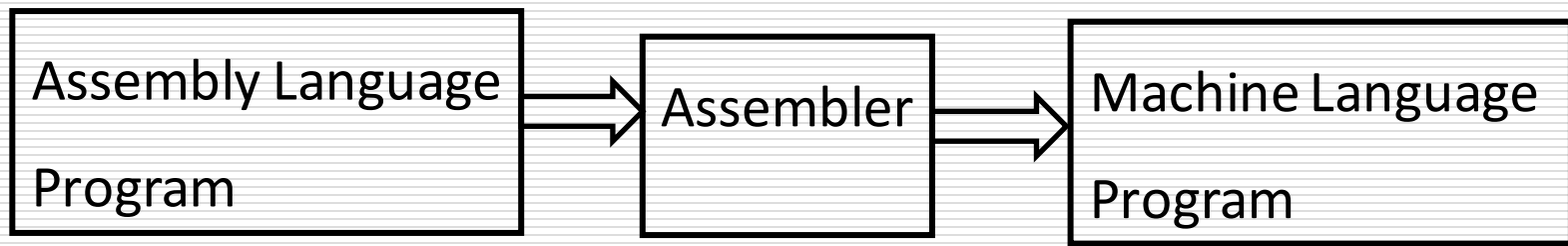
- ★ Application software is a computer software package that performs **a specific function for a user, or for another application.**
- ★ An application can be self-contained, or it can be a group of programs that run the application for the user.
- ★ Examples: office suites, graphics software, database management programs, web browsers, word processors, image editors.

Translators

- ★ A **translator** is a program that converts source code into machine code.
- ★ Any **program** written in a **high-level language/assembly** language is known as **source code**.
- ★ Computers cannot understand source code.
- ★ Before execution, source code must be translated into a form which a computer can understand

Assembler

- ★ Is a translator program
- ★ Assembles (Translates) assembly language program into machine language
- ★ Depends up on machine architecture



Assembler Functions

- ★ Convert mnemonic operation codes to machine language
- ★ Convert symbolic operands in to machine addresses
- ★ Build the machine instructions in the proper format
- ★ Convert the data constants into machine representations
- ★ Write the object program and assembly listing

Compiler

- ★ Is a translator program
- ★ Translates high level language program into its equivalent machine language
- ★ Entire program is translated into machine language before execution
- ★ Object program is stored into a separate file

Interpreter

- ★ Is a translator program
- ★ Translates high level language program into its equivalent machine language
- ★ Each high level language statement is translated and executed before translating the next statement
- ★ Object program is not stored into file

Interpreter/Compiler – Basic Functions

1. Lexical Analysis

- ★ Scans the source program, recognizing and classifying the various tokens

2. Syntax Analysis

- ★ Source program statements are recognized as language constructs described by the grammar being used

3. Object code generation

Interpreter/Compiler – Basic Functions



What is a token? A lexical token is a sequence of characters that can be treated as a unit in the grammar of the programming languages. **Example of tokens:**

- Type token (id, number, real, . . .)
- Punctuation tokens (IF, void, return, . . .)
- Alphabetic tokens (keywords)

Tokens

```
int main()
{
    // 2 variables
    int a, b;
    a = 10;
    return 0;
}
```

All the valid tokens are:

```
'int'  'main'  '('  ')'  '{'  'int'  'a'  ','  'b'  ';'
'a'  '='  '10'  ';'  'return'  '0'  ';'  '}'
```

Loader

- ★ Is a program that allocate memory and loads program into memory for execution.
- ★ Also perform relocation and linking of program modules

Loader

★ Assemblers and compilers are used to convert source code to object code. The loader will accept that object code, make it ready for execution, and helps to execute. Loader performs its task via four functions, these are as follows:

1. **Allocation:** It allocates memory for the program in the main memory.
 2. **Linking:** It combines two or more separate object programs or modules and supplies necessary information.
 3. **Relocation:** It modifies the object program so that it can be loaded at an address different from the location.
 4. **Loading:** It brings the object program into the main memory for execution.
-

Loader Functions

- ★ **Allocation:** allocate space in memory for the programs

Loader Functions

- ★ **Linking:** The linker resolves the symbolic reference code or data between the object modules by allocating all of the **user subroutine** and **library subroutine** addresses. **This process is known as linking.**
- ★ In any language, a program written has a function, it can be user-defined or can be a library function. For example, in C language we have a **printf() function**. When the program control goes to the line where the printf() is written, then the linker comes into the picture and it links that line to the module where the **actual implementation of the printf() function is written.**

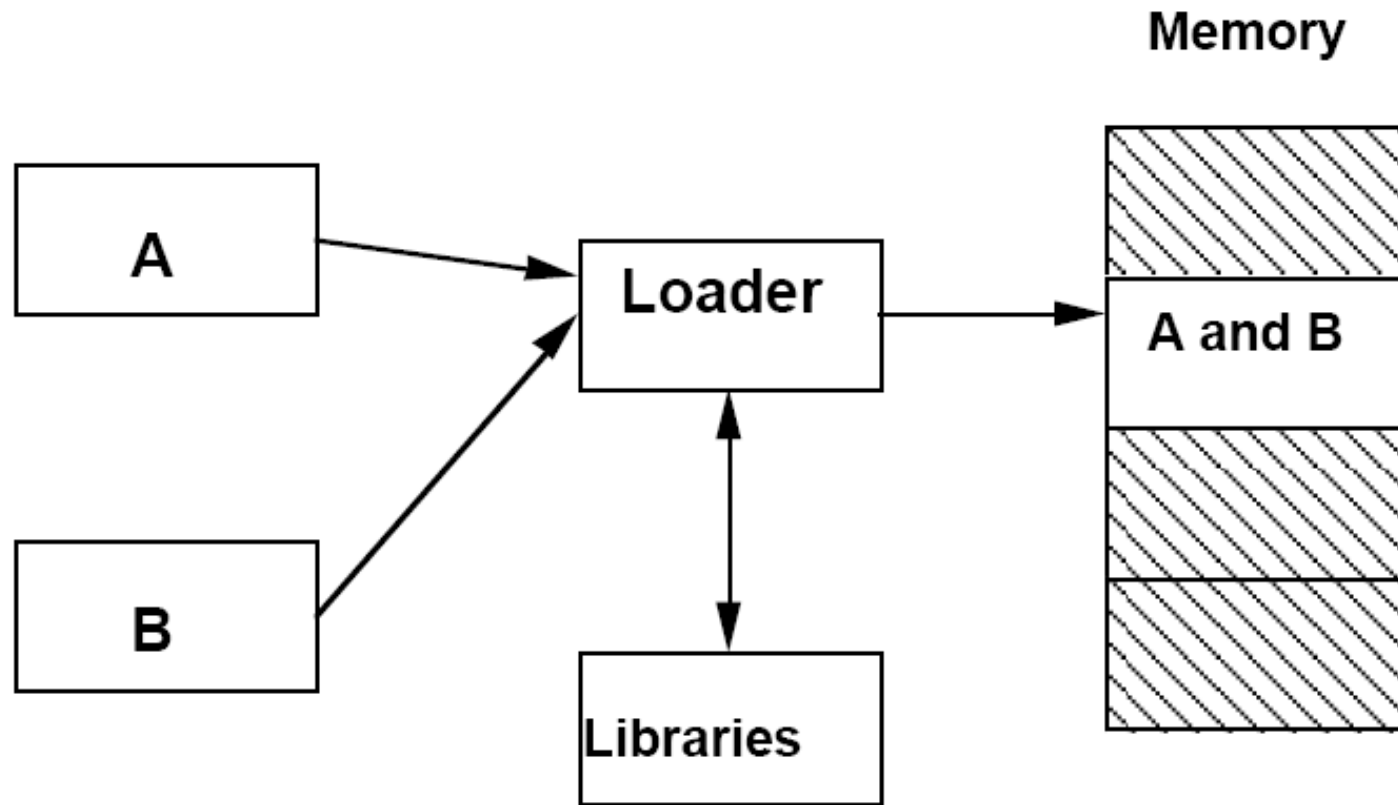
Loader Functions

★ **Relocation:** There are some **address-dependent locations in the program**, and these **address constants must be modified to fit the available space, this can be done by loader and this is known as relocation**. In order to allow the object program to be **loaded at a different address than the one initially supplied, the loader modifies the object program by modifying specific instructions**.

Loader Functions

- ★ **Loading:** Physically place the machine instructions and data into memory

Loader



Operating System & Its Functions

- ★ A program that acts as an intermediary between a user of a computer and the computer hardware
- ★ Functions of operating systems are:
 1. Manages and Interacts with Computer Hardware
 2. Resource management
 3. Provides and Manages System Security
 4. Provides the System Interface
 5. Provides the Interface for Application Software

Operating System Functions

Process Management

- ★ Process is a program in execution
- ★ A process needs certain resources – including CPU time, memory, files and I/O devices – to accomplish its task
- ★ These resources are either given to the process when it is created, or allocated to it while it is running

Process Management

- ★ OS is responsible for the following activities in connection with process management
 - ★ Creating and deleting both user and system processes
 - ★ Suspending and resuming processes
 - ★ Providing mechanism for process synchronization
 - ★ Providing mechanism for process communication
 - ★ Providing mechanisms for deadlock handling

Main Memory Management

- ★ Processor reads instructions from main memory during the instruction fetch cycle
- ★ Reads & writes data from memory during the data fetch cycle
- ★ I/O operations implemented by DMA read and write data in main memory
- ★ Main memory is the only large storage device that the CPU is able to address and access directly

Main Memory Management

- ★ When the program terminates, its memory space is declared available, and the next program can be loaded and executed
- ★ To improve both the utilization of the CPU and the speed of the computer's response, several programs must keep in memory
- ★ Many different memory management schemes are available

Main Memory Management

- ★ The operating system is responsible for the following activities in connection with memory management
 - ★ Keeping track of which parts of memory are currently being used and by whom.
 - ★ Deciding which processes are to be loaded into memory when memory space becomes available.
 - ★ Allocating and de allocating memory space as needed.

File Management

- ★ OS provides a uniform logical view of information storage
- ★ Operating system abstracts from the physical properties of its storage devices to define a logical storage unit
- ★ It maps files onto physical media, and accesses these files via the storage devices

File Management

- ★ Files are normally organized into directories to ease their use
- ★ When multiple users have access to files, we may want to control by whom and in what ways files may be accessed

File Management

- ★ OS is responsible for the following activities in connection with file management.
 - ★ Creating and deleting files
 - ★ Creating and deleting directories
 - ★ Supporting primitives for manipulating files and directories
 - ★ Mapping files onto secondary storage
 - ★ Backing up files on stable storage media.

I/O System Management

- ★ One of the purposes of an operating system is to hide the details of specific hardware devices from the user
- ★ Only the device driver knows the details of the specific device to which it is assigned

I/O System Management

- ★ The I/O subsystem consists of
 - ★ A memory management component that includes buffering, caching, and spooling
 - ★ A general device driver interface
 - ★ Drivers for specific hardware devices

Networking

- ★ A distributed system is a collection of processors that do not share memory, peripheral devices, or a clock
- ★ The processors in the system are connected through a communication network
- ★ The network may be fully or partially connected

Networking

- ★ The communication network design must consider message routing, and connection strategies, and the problem of contention and security.
- ★ Operating systems usually generalize network access as a form of file access.

Protection System

- ★ Various processes must be protected from one another's activities
- ★ Mechanisms must ensure that the files, memory segments, CPU, and other resources can be operated on by only those processes that have gained proper authorization from the operating system

Protection System

- ★ Protection is any mechanism for controlling the access of programs, processes, or users to the resources defined by a computer system
- ★ A protection oriented system provides a means to distinguish between authorized and unauthorized usage.

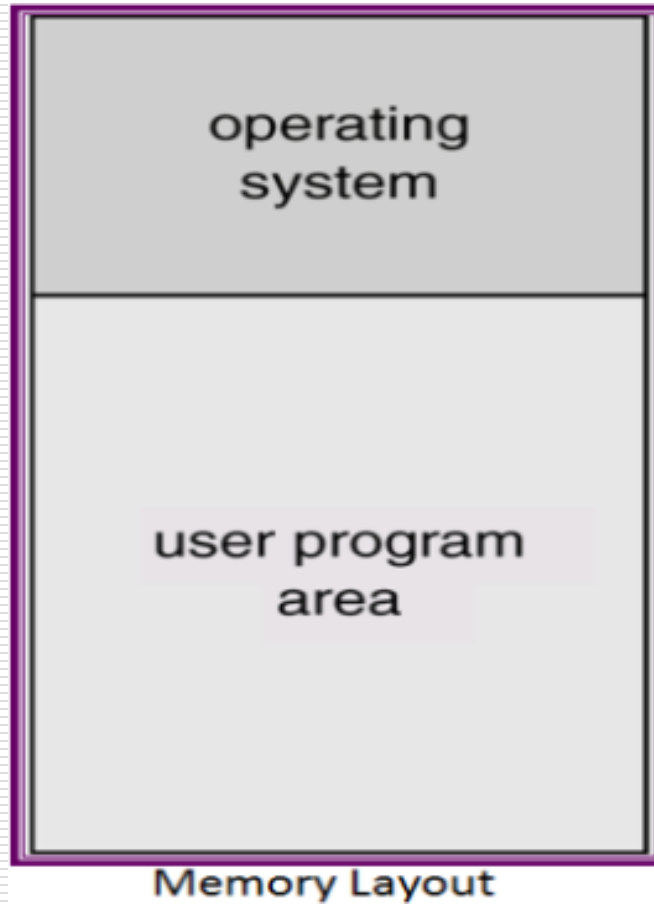
Types Of Operating System

1. Batch Systems
2. Multi programming systems
3. Time sharing systems
4. Multi processor systems
5. Real time systems

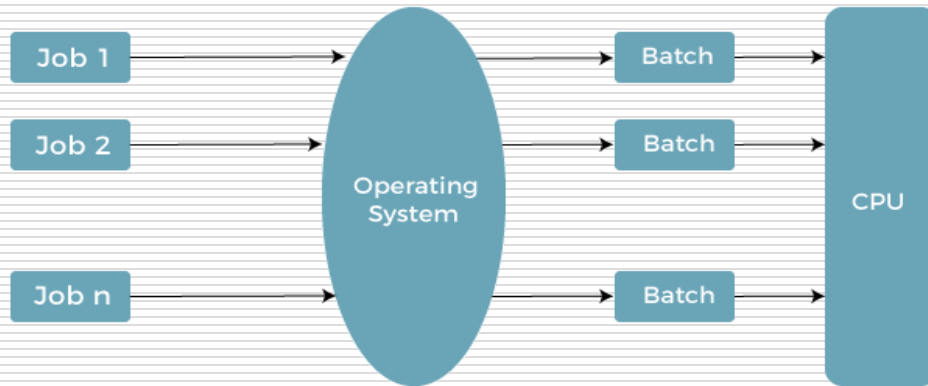
1. Batch System

- ★ The operating system was always resident in memory
- ★ Major task was to transfer control automatically from one job to the next
- ★ To speed up processing, operators batched together jobs with similar needs and ran them as a group

1. Batch System



1. Batch System

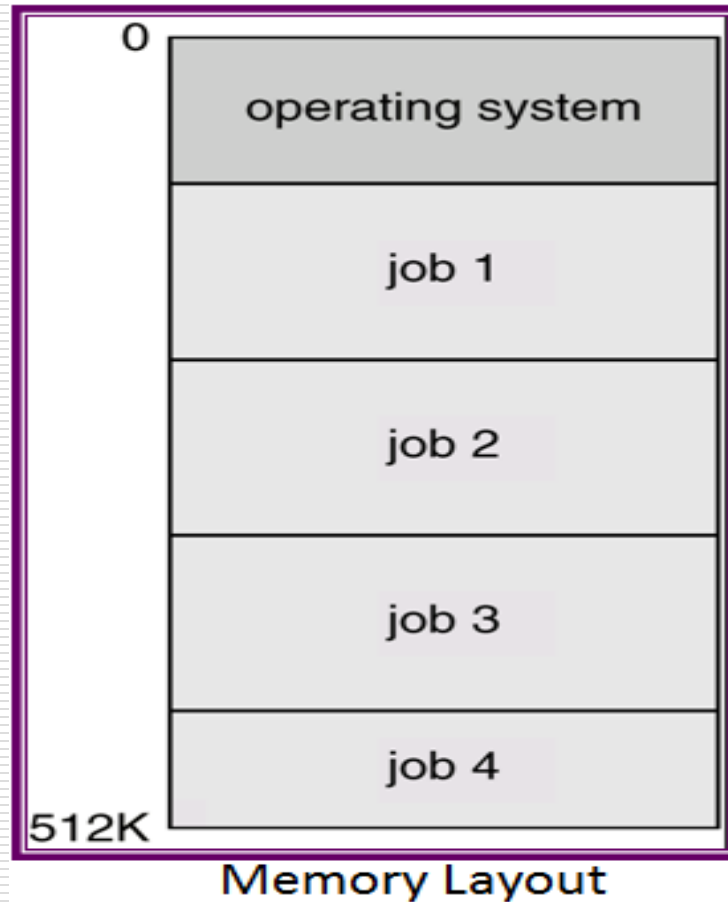


Batch processing was very popular in the **1970s**. The jobs were executed in batches. People used to have a single computer known as a **mainframe**. Users using batch operating systems do not interact directly with the computer. Each user prepares their job using an offline device like a punch card and submitting it to the computer operator. Jobs with similar requirements are grouped and executed as a group to speed up processing. Once the programmers have left their programs with the operator, they sort the programs with similar needs into batches.

2. Multiprogrammed System

- ★ The operating system keeps several jobs in memory simultaneously
- ★ The operating system picks and begins to execute one of the jobs in the memory
- ★ When the job may have to wait for some task, the operating system simply switches to, and executes another job.

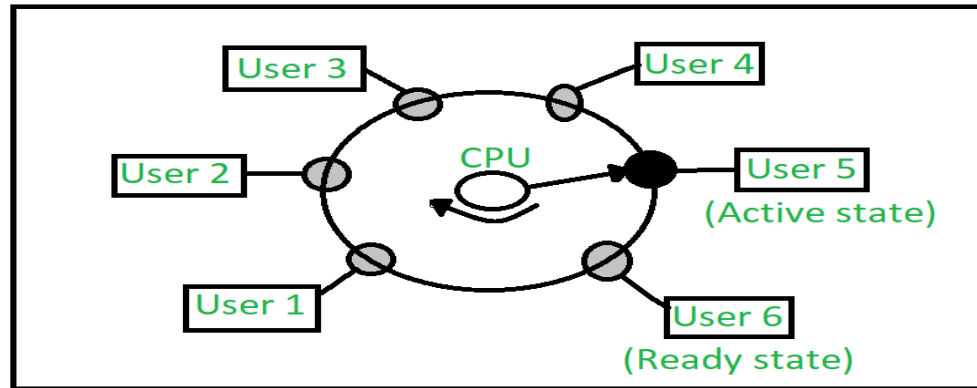
2. Multiprogrammed System



3. Time-Sharing System

- ★ The CPU executes multiple jobs by switching among them
- ★ The switches occur so frequently that the user can interact with each program while it is running
- ★ A time shared operating system allows many users to share the computer simultaneously
- ★ A time shared operating system uses CPU scheduling and multiprogramming
- ★ Each user has at least one separate program in memory

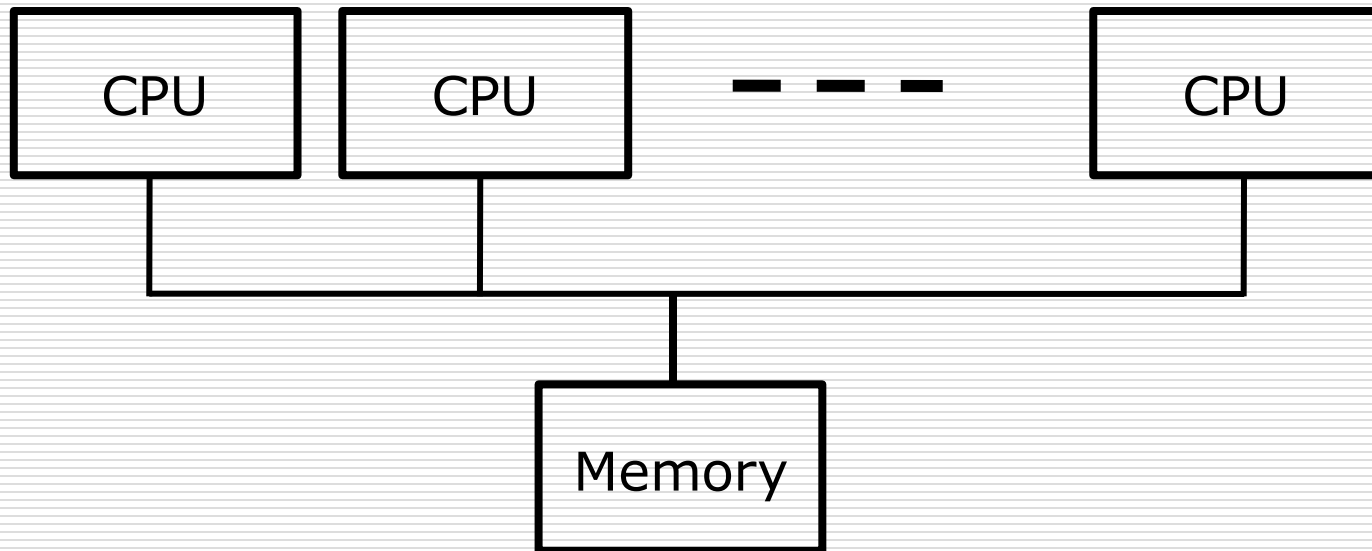
3. Time-Sharing System



A **time-shared operating system** uses CPU scheduling and multi-programming to provide each user with a small portion of a shared computer at once. Each user has at least one separate program in memory. A program is loaded into memory and executes, it performs a short period of time either before completion or to complete I/O. This short period of time during which the user gets the attention of the CPU is known as **time slice, time slot, or quantum**. It is typically of the order of 10 to 100 milliseconds. Time-shared operating systems are more complex than multiprogrammed operating systems. In both, multiple jobs must be kept in memory

4. Multiprocessor Systems

- ★ Multiprocessor systems have more than one processor in close communication, sharing the computer bus, the clock, and sometimes memory and peripheral devices.



4. Multiprocessor Systems

★ Advantages

1. **Increased throughput** : Get more work done in less time.
2. **Economy of scale** : They can share peripherals, mass storage, and power supplies.
3. **Increased reliability** : The failure of one processor will not halt the system, only slow it down

4. Multiprocessor Systems

★ Two configurations

1. **Symmetric multiprocessing (SMP)** – Each processor runs an identical copy of the operating system, and these copies communicate with one another as needed.
2. **Asymmetric multiprocessing** - Each processor is assigned a specific task. A master processor schedules and allocated work to slave processors.

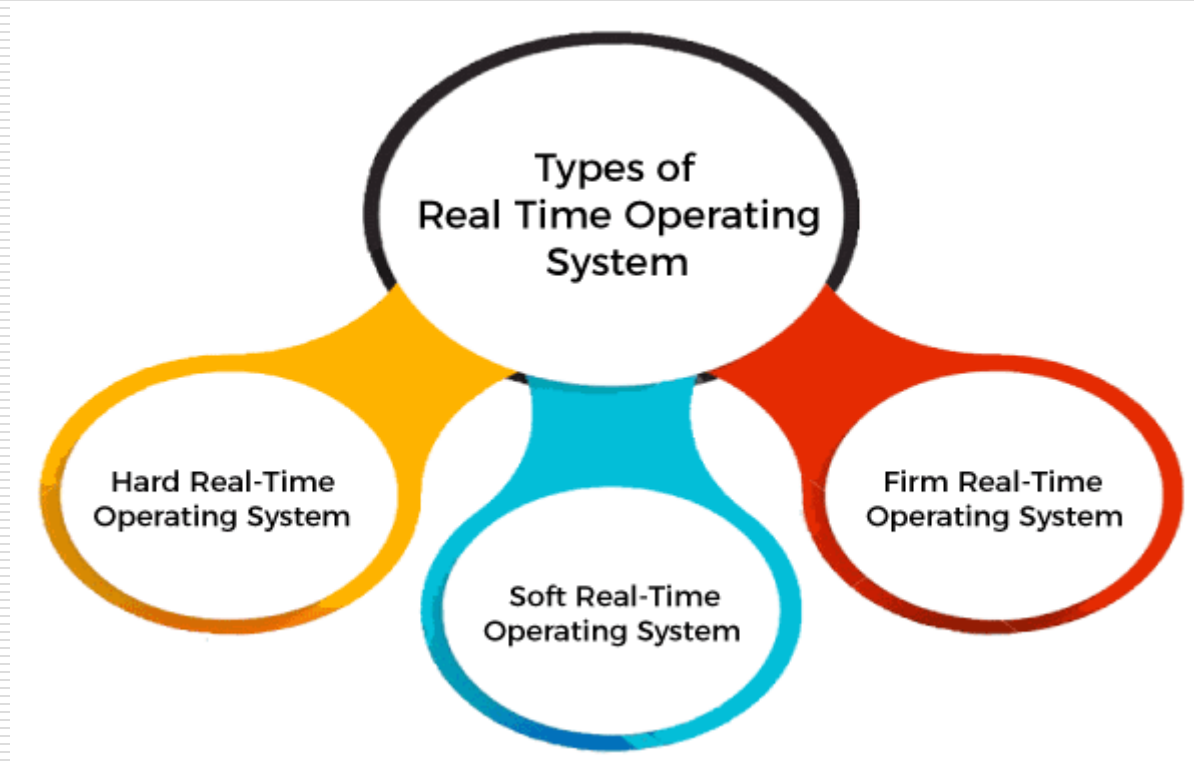
5. Real Time Systems

- ★ **A real-time operating system (RTOS)** is a special-purpose operating system used in computers that has strict time constraints for any job to be performed. It is employed mostly in those systems in which the results of the computations are used to influence a process while it is executing. Whenever an event external to the computer occurs, it is communicated to the computer with the help of some sensor used to monitor the event. The sensor produces the signal that is interpreted by the operating system as an interrupt. On receiving an interrupt, the operating system invokes a specific process or a set of processes to serve the interrupt.

5. Real Time Systems

- ★ A real time system is used when rigid time requirements have been placed on the operation of a processor or the flow of data
- ★ Real time system has well defined fixed time constraints
- ★ Processing must be done within the defined constraints, or the system will fail

5. Real Time Systems



5. Real Time Systems

★ Two flavors

1. **Hard Real Time System** - Guarantees that critical task be completed on time
2. **Soft Real Time System** – Critical task gets priority over other task, and retains that priority until it completes.